

DEEP LEARNING INDABA · SKILLS SESSION

Building On-Device AI Applications

with Gemma 3n (...or pick your own fighter)

Jesse-Paul Osemeke

“Sovereign Intelligence: Africa’s Path in a Frontier AI World”

What we'll cover today

01

On-device model selection

What makes a model suitable for local deployment — size, speed, efficiency, hardware fit.

03

Practical conversion workflow

Step-by-step, using prewritten scripts to prepare models for local inference.

05

Sovereign AI

How on-device AI supports Africa's technical independence, privacy, and local capacity.

02

Model quantisation

How it works, what it changes and saves, and the key algorithms: GPTQ, AWQ, K-quants.

04

Common pitfalls & debugging

What goes wrong deploying on laptops and desktops — and how to fix it.

One app, fully local

Anya — Research Assistant

A local-first desktop app for running literature reviews and brainstorming ideas.
Powered by a quantised LLM, fully offline — no data leaves your machine.

STACK

Python · Ollama · Gemma 3n

github.com/floatinggonion/anya-ra

● Running locally

WHY IT MATTERS

- Fully offline — no API, no cloud
- Runs on consumer hardware
- Literature review & brainstorming
- Powered by quantised Gemma 3n

What makes a model on-device ready?



Low memory footprint

Must fit in device RAM — typically 2–6 GB on phones, 8–16 GB on laptops.



High inference speed

Low FLOPs; generates tokens fast enough to feel real-time.



Energy efficiency

Low power draw — critical for mobile, to avoid draining batteries.



Hardware acceleration

Leverages on-chip NPUs, GPUs, or Apple Neural Engine.



Quantisation readiness

Retains accuracy even when weights are heavily compressed.

These five criteria filter which models survive compression — Gemma 3n was built with exactly these constraints in mind.

Why Gemma 3n?

- Google's on-device-first multimodal model family
- Designed from the ground up for mobile and edge deployment
- Multimodal: accepts text, audio, and image inputs
- Flexible compute budget via Per-Layer Embedding (PLE) architecture
- Quantised weights available on Hugging Face out of the box
- Comparable to Qwen3.5 0.8B as an ultra-lightweight alternative

Gemma 3n E2B

~2B params · ~4 GB RAM

~3.31 GB quantised (Q4_K_M)

Mobile-friendly

Gemma 3n E4B

~4B params · ~6–8 GB RAM

Qwen3.5 0.8B → lightweight alt.

Laptop-class

PART 2 OF 5

Model Quantisation

What it changes · What it saves · The trade-offs

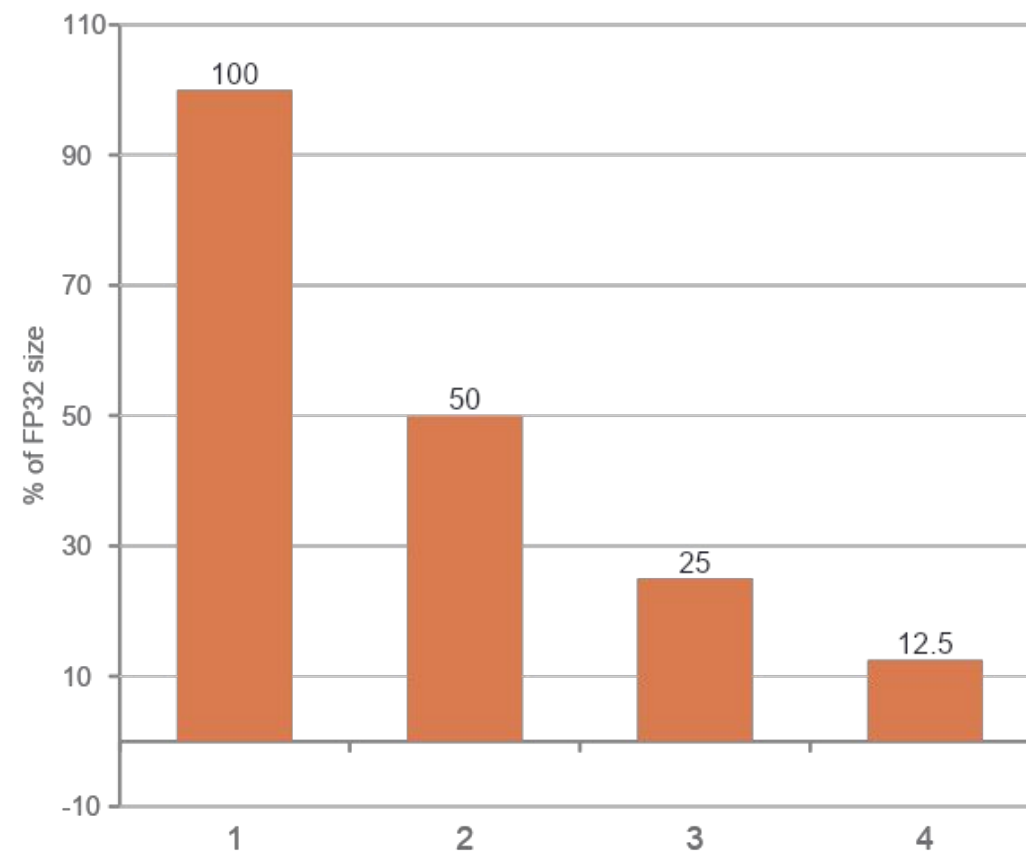
What is quantisation?

Reducing the numerical precision of a model's weights — shrinking size and memory use while preserving most of its capability.

THE ANALOGY

Compressing audio from WAV to MP3. You lose some ultra-high-frequency detail most ears can't hear — but the song still sounds great, and the file is 10× smaller. Quantisation trades a sliver of precision for a massive drop in size and compute.

Precision → size



Anatomy of a float

Every float splits its bits into three jobs. The sign says +/-. The exponent sets the range. The mantissa sets the precision.

$$\text{value} \approx (-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exponent} - \text{bias})}$$

■ sign

■ exponent — range

■ mantissa — precision

FP32		<table><tr><th>Format</th><th>Bits</th><th>Max magnitude</th><th>Precision</th></tr><tr><td>FP32</td><td>32</td><td>$\pm 3.4 \times 10^{38}$</td><td>~7 digits</td></tr><tr><td>FP16</td><td>16</td><td>$\pm 65,504$</td><td>~3 digits</td></tr><tr><td>BF16</td><td>16</td><td>$\pm 3.39 \times 10^{38}$</td><td>~2 digits</td></tr><tr><td>INT4</td><td>4</td><td>-8 to 7</td><td>exact (16 buckets)</td></tr></table>	Format	Bits	Max magnitude	Precision	FP32	32	$\pm 3.4 \times 10^{38}$	~7 digits	FP16	16	$\pm 65,504$	~3 digits	BF16	16	$\pm 3.39 \times 10^{38}$	~2 digits	INT4	4	-8 to 7	exact (16 buckets)
Format	Bits		Max magnitude	Precision																		
FP32	32		$\pm 3.4 \times 10^{38}$	~7 digits																		
FP16	16		$\pm 65,504$	~3 digits																		
BF16	16		$\pm 3.39 \times 10^{38}$	~2 digits																		
INT4	4	-8 to 7	exact (16 buckets)																			
FP16																						
BF16																						
INT4	plain signed integer — no exponent needed																					

The quantisation formula — symmetric

QUANTIZE

scale = max(|values|) / q_max
q = round(v / scale)

DEQUANTIZE

v' = q × scale

One scale factor maps your float range onto the integer range, centred on zero.
Simple — but wasteful if your data isn't symmetric.

~18%

Average dequantised error — 6 real weights, symmetric int4 (q_max = 7, scale ≈ 0.127)

Original	Quantised	Dequantised	Error
-0.89	-7	-0.890	0.0%
0.16	1	0.127	-20.6%
0.08	1	0.127	+58.9%
-0.13	-1	-0.127	-2.2%
0.16	1	0.127	-20.6%
-0.54	-4	-0.509	-5.8%

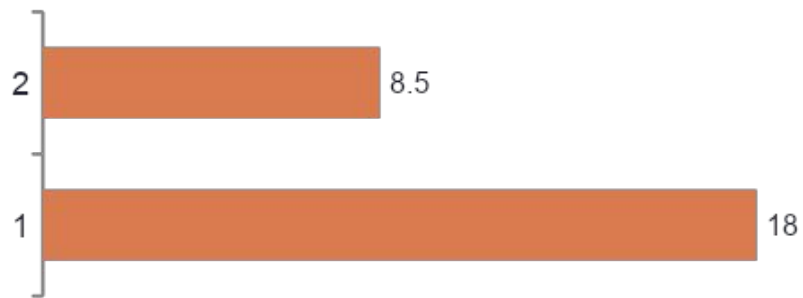
Asymmetric quantisation — adding a zero-point

```
QUANTIZE
scale = (max - min) / (q_max - q_min)
zero = q_min - round(min / scale)
q = round(v / scale + zero)

DEQUANTIZE
v' = scale × (q - zero)
```

Squishes around the midpoint of your data instead of zero, using the full integer range — same 4 bits, roughly half the error.

Same 4 bits, half the error



Original	Quantised	Dequantised	Error
-0.89	-8	-0.91	+2.2%
0.16	7	0.14	-12.5%
0.08	6	0.07	-12.5%
-0.13	3	-0.14	+7.7%
0.16	7	0.14	-12.5%
-0.54	-3	-0.56	+3.7%

$scale \approx 0.07 \cdot zero = 5$

QUANTISATION

Why is floating point so expensive?

Adding two FP numbers isn't simple addition — it's a four-step hardware pipeline.



FP32 / FP16: 4-step pipeline per operation, multiple clock cycles, complex hardware logic. **INT8:** direct bit summation — no alignment, no normalisation. Fewer transistors, less power, higher throughput.

Quantisation usually skips the embedding layer and LM head to preserve accuracy — it targets the internal feed-forward and attention blocks.

Two main quantisation settings

W8A8

Weight + activation quantisation

Both weights and activations quantised to INT8.

ADVANTAGES

- Reduces both memory AND compute
- Full integer arithmetic at runtime
- Strong throughput on INT8 hardware

WATCH OUT FOR

- Activations vary at runtime — harder to quantise
- Needs calibration data

W4A16

Weight-only quantisation

Weights compressed to INT4; activations stay FP16.

ADVANTAGES

- Up to 75% memory saving
- Better accuracy retention than W8A8
- Most common for on-device deployment

WATCH OUT FOR

- Weights dequantised at runtime — small overhead
- Less throughput gain vs W8A8

Core approaches: PTQ vs QAT

PTQ

Post-training quantisation

Applied to an already-trained checkpoint. No retraining required.

KEY ADVANTAGES

- Works on any pre-trained model
- Fast to apply — minutes to hours
- The approach used in this session
- Go-to for on-device deployment

QAT

Quantisation-aware training

Quantisation errors are simulated during training so the model learns to compensate.

KEY ADVANTAGES

- Higher accuracy at very low bit widths
- Better for INT4 / INT2 scenarios
- Used in production fine-tuning
- Expensive but more accurate

This session uses PTQ throughout — fast, no retraining, good results for on-device deployment.

Key algorithms: GPTQ & AWQ



GPTQ

Layer-by-layer, error-minimising

Generative Pre-trained Transformer Quantisation

Processes the model layer by layer. For each layer, finds the optimal rounding of weights that minimises output error.

FP16 → 3–4 bits, up to 75% memory reduction, ~1–2% perplexity loss.



AWQ

Salience-aware, protects critical weights

Activation-Aware Weight Quantisation

Key insight: ~1% of weights are 'salient' — they disproportionately swing model output. AWQ finds them via activation magnitudes and protects them.

Often better accuracy than GPTQ at the same bit width, especially INT4.

Blockwise K quantisation (K-quants)

llama.cpp's native quantisation method — the one behind Q4_K_M, Q5_K_M, Q6_K and other variants.

1 Block splitting

Weights are split into small blocks, each with its own local scale factor — much finer-grained precision control than one global scale.

2 Super-blocks

Groups of blocks bundle into 'super-blocks'. Their scale constants get a second, lighter round of quantisation, compressing the overhead.

3 Importance matrix

An imatrix, computed from sample data, identifies the most salient weights and minimises rounding error exactly there.

NAMING CONVENTION

Q4_K_M = 4-bit · K-quant · Medium quality. **Q4_K_S** = Small, **Q5_K_M** = 5-bit Medium. The **_K** suffix always means K-quant — these variants consistently beat legacy Q4_0 at the same bit depth. Always prefer them.

Why quantise?

75%

Memory saved

FP16 → INT4

4×

Faster throughput

vs. full precision

0

Cloud calls

Fully offline & private



Energy per token

Less battery drain



Context length

More tokens per GB RAM

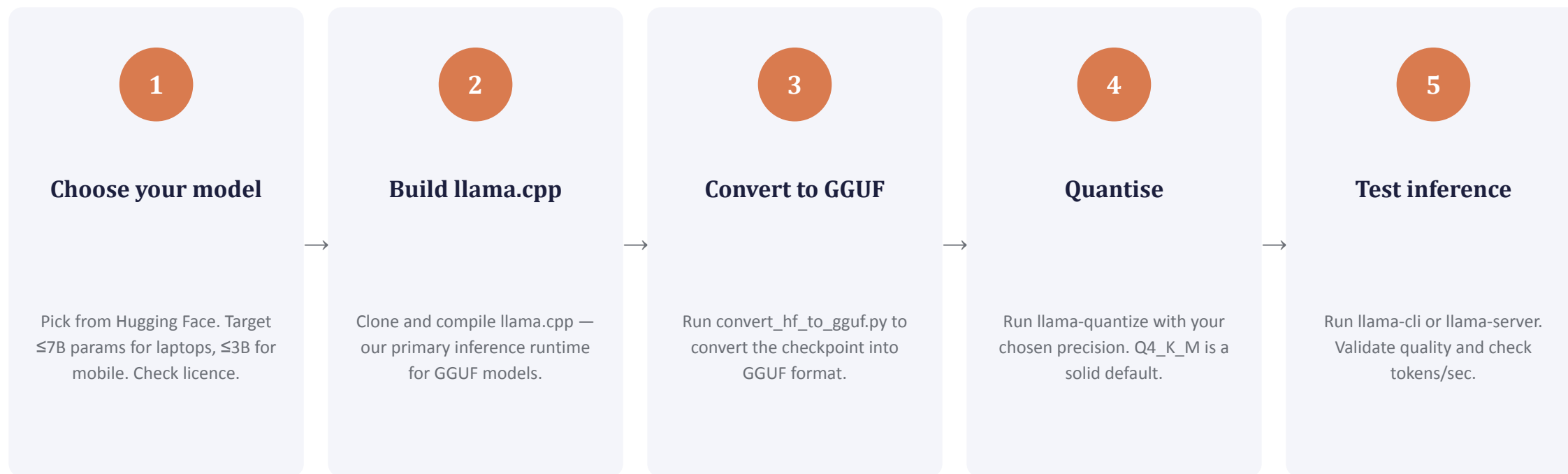
A 75% memory cut turns a 10 GB model needing a rented A100 into a 2.5 GB model that runs on a MacBook. 'Zero cloud calls' is the number that carries the sovereign-AI story.

PART 3 OF 5

Practical Conversion Workflow

From Hugging Face to local inference — step by step

Preparing a model for local inference



Pre-quantised models: huggingface.co/FloatingOnion/mobile-llm-quant-demo

Edge & mobile inference frameworks

llama.cpp / GGUF

macOS · Linux · Windows · Android

Go-to runtime for quantised LLMs on CPU and GPU. GGUF is the primary format used in this session.

ONNX Runtime

Cross-platform · Mobile · Browser

Wide hardware support via optimised execution providers. Good for cross-platform edge deployment.

TensorFlow Lite

Android · iOS · Embedded

Mature mobile runtime, best for smaller task-specific models (classification, vision, audio).

Apple Core ML

iOS · macOS

Apple's native framework, leveraging the Neural Engine on A-series / M-series chips.

This session uses llama.cpp / GGUF — the most versatile and best maintained option, working on any OS.

PART 4 OF 5

Common Pitfalls & How to Debug Them

What goes wrong — and what to do about it

Common pitfalls — and how to fix them



Out of memory (OOM) during conversion

Use a machine with ≥ 16 GB RAM. Start with a smaller model ($\leq 3B$). Offload layers to disk with `--offload-kqv`.



Gibberish output after quantisation

Tokeniser likely wasn't converted correctly. Re-run GGUF conversion and check tokeniser files are included.



GGUF version mismatch (model won't load)

Update llama.cpp to the latest build — GGUF format versions change frequently; old files often unsupported.



Very slow inference on CPU (>10 sec/token)

Pass `-ngl 99` to offload layers to GPU if available. On CPU-only, 1–4 tok/s for 3–4B models is normal.



Noticeable quality loss after quantisation

Move from Q4_0 to Q4_K_M — the _K variants group better. Or go up to Q5 / Q6 for more accuracy.

Sovereign Intelligence

Data stays local

Patient records, government documents, personal data — none of it leaves the device. Privacy is architecture, not policy.

No cloud dependency

No API bills. No foreign infrastructure. No outages from a distant data centre. AI that works anywhere, even offline.

Adaptable to local context

Local models can be fine-tuned on local hardware for African languages, dialects, and domain knowledge.

Build, don't just consume

These skills put deployment power in African hands — to prototype, ship, and iterate without gatekeepers.

WHAT'S NEXT

Next steps & resources

YOUR CHECKLIST

- ☐ Install llama.cpp on your machine
- ☐ Download a pre-quantised model from the repo
- ☐ Run inference locally — confirm it works
- ☐ Convert and quantise a model of your choice
- ☐ Evaluate output quality (compare to original)
- ☐ Build something useful with it

RESOURCES

Session repo (scripts & notebook)

github.com/FloatingOnion/dl-indaba-resources

Shared drive (slides & extras)

drive.google.com/drive/folders/1hwQdn33z...

Pre-quantised models (HF)

hf.co/FloatingOnion/mobile-llm-quant-demo

Quantisation in depth (DLAI)

deeplearning.ai/courses/quantization-in-depth

Quantisation fundamentals (HF×DLAI)

deeplearning.ai/courses/quantization-fundamentals

A visual guide to quantisation

newsletter.maartengrootendorst.com

AWQ & GPTQ papers (arXiv)

arxiv.org/pdf/2306.00978 · arxiv.org/abs/2210.17323

AWQ & GPTQ papers (arXiv)

<https://ngrok.com/blog/quantization>

APPRECIATION

With thanks



Daniel Onyesoh

With gratitude for the collaboration, insight, and hands-on work that helped shape this session — from the demos to the ideas behind them.

Thank you

Questions?

Jesse-Paul Osemeke

Repo: github.com/FloatingOnion/dl-indaba-resources

Drive: drive.google.com/drive/folders/1hwQdn33z...

Questions? jesseosems123@gmail.com

“The path to sovereign intelligence is built one model at a time.”